



SECURITY REVIEW • PREPARED FOR

HoodLock

Uniswap v3 and v4 liquidity locking on
Robinhood Chain

DATE

04.10.2026

COMMIT

5af08550

REMEDIATION

634480d0

Phase & process

1. About SBSecurity

SBSecurity is a team of skilled smart contract security researchers. Based on the audits conducted and numerous vulnerabilities reported, we strive to provide the absolute best security service and client satisfaction. While it's understood that 100% security and bug-free code cannot be guaranteed by anyone, we are committed to giving our utmost to provide the best possible outcome for you and your product.

Book a Security Review with us at sbsecurity.net or reach out on Telegram [@sbsecurity_audits](https://t.me/sbsecurity_audits).

2. Disclaimer

A smart contract security review can only show the presence of vulnerabilities **but not their absence**. Audits are a time, resource, and expertise-bound effort where skilled technicians evaluate the codebase and their dependencies using various techniques to find as many flaws as possible and suggest security related improvements. We as a company stand behind our brand and the level of service that is provided but also recommend subsequent security reviews, on-chain monitoring, and high whitehat incentivization.

3. Risk classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

3.1 IMPACT

- **High** - leads to a significant loss of assets in the protocol or significantly harms a group of users.
- **Medium** - leads to a moderate loss of assets in the protocol or some disruption of the protocol's functionality.
- **Low** - funds are not at risk.

3.2 LIKELIHOOD

- **High** - almost certain to happen, easy to perform, or highly incentivized.
- **Medium** - only conditionally possible, but still relatively likely.
- **Low** - requires specific state or little-to-no incentive.

02 EXECUTIVE SUMMARY

Key findings

TOTAL FINDINGS

6

RESOLVED

6 of 6 · 100%

HIGH SEVERITY FINDINGS

0 none found

SEVERITY DISTRIBUTION

☒ Fixed & verified ☒ Acknowledged

CRITICAL 0

HIGH 0

MEDIUM 2

LOW 3

INFORMATIONAL 1

GAS OPT. 0

CLIENT

HoodLock

REPOSITORY

Private

METHODS

Manual review

CONTRACTS IN SCOPE

src/HoodLockLPLocker.sol

TIMELINE · OCTOBER 2026

Audit · 2 days

Remediation · 1 day

KICK-OFF

03.10.2026

END OF AUDIT

04.10.2026

REMEDIATIONS START

04.10.2026

REMEDIATIONS END

04.10.2026

STARTING COMMIT HASH

5af08550d606d6f7a458abbe1e7c09915df27def

FINAL COMMIT HASH

634480d01f9793cbc294a7f47aa6a3b129ae64b9

03 PROJECT SUMMARY

What HoodLock is

HoodLock's LP locker is a single contract on Robinhood Chain that holds Uniswap v3 and v4 position NFTs until an unlock date the owner picks. While a position is locked, the owner keeps collecting its trading fees and HoodLock takes a share of each collect, fixed per lock when it is created. At the unlock date the owner withdraws the NFT in one transfer that never touches the pool, a token or a hook.

SECURITY POSTURE

BEFORE REMEDIATION

5AF08550

AFTER REMEDIATION

634480D0

Fee collection

M-01 · M-02 · L-02

Three things could make a lock's collect revert: a fee recipient that refused an ERC-20 share, a v4 hook that took a cut on removal and so failed the PositionManager slippage check, and a token that returned malformed data on transfer. The fees then stayed in the position until unlock, and since withdraw did not settle them, the HoodLock cut on them was lost.

Admin key scope

L-01

`adminCollect` was admin-only, so a bot running it on a schedule needed the full admin key.

Owed ETH

L-03

ETH credited to a recipient that could not receive ETH, such as a vesting or escrow contract that took over a lock, could only be paid to that same address and stayed stuck.

Lock status

I-01

`isUnlocked` returned true for lock ids that were never created.

All six findings were fixed by the final commit, each in its own commit. An ERC-20 share the fee recipient refuses now goes to the lock owner instead of reverting, and v4 fees are collected with a zero-liquidity increase, so a hook's cut on removal comes out of the fees. Malformed transfer return data counts as a failed transfer, a credited address can send its owed ETH elsewhere with `claimOwedEthTo`, and `isUnlocked` reverts for ids that were never created. `adminCollect` can also be run by a new keeper role that has no other power. SB Security reviewed each fix against its issue, and the team added fork tests that cover every finding.

04 RECOMMENDATIONS

Beyond this audit

Measures that extend the protection of this review after launch, independent of any individual finding.

R-01	Keep admin keys under strong custody	Hold the admin role in a hardware-backed multisig, give the keeper bot its own key, and announce fee and allowlist changes before making them.
R-02	Review hooks before allowlisting	Check a hook's permissions, fee behaviour and upgrade path before adding it to the allowlist, and track changes to the hooks already on it.
R-03	Monitor locks on chain	Index the Collected, CollectFailed and ShareRedirected events and alert on repeated collect failures, parked cuts and unclaimed ETH credits, so the team sees an unusual lock early.
R-04	Keep the fix tests in CI	Run the fork suite, including the tests added with the fixes, on every change to the locker, and treat a failure as a release blocker.

Core contracts

CONTRACT	ROLE
HoodLockLPLocker.sol	The only contract in scope. Holds Uniswap v3 and v4 position NFTs until their unlock date, collects their fees for each lock's fee recipient, sends the HoodLock cut to the fee vault, and returns the NFT to the owner on withdraw.

Privileged roles

Admin.

Sets the collect fee and the fixed fee for new locks within bounds fixed in the contract, the fee vault and revenue addresses, and the hook allowlist, and can pause new locks. It can also run `adminCollect`, name the keeper, return stray tokens and position NFTs that are not part of a lock, and hands over the role in two steps. It cannot move a locked position or change an existing lock.

Keeper.

Named by the admin. Can run `adminCollect` and nothing else, and funds go exactly where the lock owner's own collect would send them.

Lock owner.

Collects the lock's fees, extends its unlock date, sets its fee recipient, transfers the lock and withdraws the NFT once it unlocks.

07 FINDINGS

What we found

ID	TITLE	SEVERITY	STATUS
M-01	Lock owners can avoid the collect fee	MEDIUM	● FIXED
M-02	Fees cannot be collected from v4 pools whose hook takes a cut on removal	MEDIUM	● FIXED
L-01	<code>adminCollect</code> requires the full admin key	LOW	● FIXED
L-02	<code>_tryTransfer</code> can revert instead of returning <code>false</code>	LOW	● FIXED
L-03	Owed ETH is stuck if the recipient cannot receive ETH	LOW	● FIXED
I-01	<code>isUnlocked</code> returns true for non-existent locks	INFORMATIONAL	● FIXED

● MEDIUM SEVERITY · 2 FINDINGS

M-01

MEDIUM

● FIXED

Lock owners can avoid the collect fee

The collect fee is only taken when fees are collected through the locker. `withdraw` returns the NFT without collecting, so after unlock the owner can collect the remaining fees from Uniswap directly, with no cut:

<https://github.com/Hoodlock/hoodlock-lp-locker-audit/blob/5af08550d606d6f7a458abbef7c09915df27def/src/HoodLockLPLocker.sol#L421-L432>

```
function withdraw(uint256 id, address to) external nonReentrant onlyLockOwner(id) {
    ...
    IERC721Min(manager).transferFrom(address(this), to, l.tokenId); // @audit fees are not
    settled
    emit Withdrawn(id, to);
}
```

`adminCollect` lets the protocol collect before unlock, but the owner can block it. The owner share is paid with `_send`, and a failed ERC-20 transfer reverts the whole collect:

<https://github.com/Hoodlock/hoodlock-lp-locker-audit/blob/5af08550d606d6f7a458abbef7c09915df27def/src/HoodLockLPLocker.sol#L576-L578>

```
} else {
    require(_tryTransfer(token, to, amount), "transfer failed");
}
```

If the pair has a token with a recipient blacklist (for example USDC), the owner sets `feeRecipient` to a blacklisted address. Every collect of the lock then reverts, `adminCollect` only emits `CollectFailed`, and at unlock the owner withdraws and takes all the fees.

IMPACT

The protocol loses its collect fee (25% by default) on all fees of such a lock. On other pairs, it loses the fee on everything earned since the last `adminCollect`. User funds are not at risk.

RECOMMENDATION

In `_collect`, if the ERC-20 owner share cannot be sent to `feeRecipient`, send it to the lock owner instead of reverting, and emit an event for it. A refused recipient then cannot block `adminCollect`:

<https://github.com/Hoodlock/hoodlock-lp-locker-audit/blob/5af08550d606d6f7a458abbef7c09915df27def/src/HoodLockLPLocker.sol#L352-L356>

```
- _send(m.token0, m.feeRecipient, got0 - cut0);
- _send(m.token1, m.feeRecipient, got1 - cut1);
+ _sendShare(m.token0, m.feeRecipient, m.owner, got0 - cut0);
+ _sendShare(m.token1, m.feeRecipient, m.owner, got1 - cut1);
    ...
}

+ function _sendShare(address token, address to, address owner_, uint256 amount) private {
+     if (token != address(0) && amount > 0 && _tryTransfer(token, to, amount)) return;
+     _send(token, token == address(0) ? to : owner_, amount);
+ }
```

Also run `adminCollect` on each lock shortly before its `unlockTime`, so the fees earned during the lock are cut before `withdraw`.

M-02

MEDIUM

● FIXED

Fees cannot be collected from v4 pools whose hook takes a cut on removal

`_pullFees` collects v4 fees with a `DECREASE_LIQUIDITY` of 0 liquidity, followed by `TAKE_PAIR` :

<https://github.com/Hoodlock/hoodlock-lp-locker-audit/blob/5af08550d606d6f7a458abbe1e7c09915df27def/src/HoodLockLPLocker.sol#L366-L368>

```
bytes memory actions = abi.encodePacked(V4_DECREASE_LIQUIDITY, V4_TAKE_PAIR);
bytes[] memory params = new bytes[](2);
params[0] = abi.encode(l.tokenId, uint256(0), uint128(0), uint128(0), bytes(""));
```

In v4, a change of 0 liquidity runs the pool's remove-liquidity hooks. A hook with the `afterRemoveLiquidityReturnDelta` permission can keep part of the fees. The `PositionManager` then checks `liquidityDelta - feesAccrued`. When no liquidity is removed, this value equals minus the hook's cut:

<https://github.com/Uniswap/v4-periphery/blob/9969eec44cfd07e24b41de47f40276a58401976/src/PositionManager.sol#L349-L350>

```
// Slippage checks should be done on the principal liquidityDelta which is the liquidityDelta -
// feesAccrued
(liquidityDelta - feesAccrued).validateMinOut(amount0Min, amount1Min); // @audit negative if
// the hook took a cut
```

`validateMinOut` casts it with `toUint128`, which reverts on a negative value before `amount0Min` is compared:

<https://github.com/Uniswap/v4-periphery/blob/9969eec44cfd07e24b41de47f40276a58401976/src/libraries/SlippageCheck.sol#L24-L25>

```
// This means this contract will NOT support pools where the hook returns a negative delta on
// burn/decrease.
if (delta.amount0().toUint128() < amount0Min) {
```

So `collect` reverts whenever the hook takes a cut. `lock` also pulls pending fees with `_pullFees`, so it reverts for such a position that already has fees.

IMPACT

Users can lock any v4 pool by accepting the unverified hook warning. If that pool's hook takes a cut on removal, the owner gets no fees until unlock, which can be years away. `adminCollect` skips the lock too, so after `withdraw` the owner collects all the fees without the HoodLock cut. Principal and `withdraw` are not affected.

RECOMMENDATION

Collect with an `INCREASE_LIQUIDITY` of 0 liquidity and max bounds. For a change of 0, the pool call and hooks are the same as for a decrease, so the same fees arrive, and add-liquidity hooks never run. The increase path checks with `validateMaxIn`, which accepts the negative value. `TAKE_PAIR` still reverts if a hook takes more than the fees, so the locker never pays:

<https://github.com/Hoodlock/hoodlock-lp-locker-audit/blob/5af08550d606d6f7a458abbe1e7c09915df27def/src/HoodLockLPLocker.sol#L90-L96>

```
- uint8 private constant V4_DECREASE_LIQUIDITY = 0x01;
+ uint8 private constant V4_INCREASE_LIQUIDITY = 0x00;
...
```

M-02 · CONTINUED

```
- /// v3 `collect` only moves tokensOwed, v4 decreases by exactly zero.
+ /// v3 `collect` only moves tokensOwed, v4 increases by exactly zero.
...
- bytes memory actions = abi.encodePacked(V4_DECREASE_LIQUIDITY, V4_TAKE_PAIR);
+ bytes memory actions = abi.encodePacked(V4_INCREASE_LIQUIDITY, V4_TAKE_PAIR);
  bytes[] memory params = new bytes[](2);
- params[0] = abi.encode(l.tokenId, uint256(0), uint128(0), uint128(0), bytes(""));
+ params[0] = abi.encode(l.tokenId, uint256(0), type(uint128).max, type(uint128).max,
  bytes(""));
```

● LOW SEVERITY · 3 FINDINGS

L-01

LOW

● FIXED

adminCollect requires the full admin key

adminCollect is onlyAdmin, so it uses the same key that controls feeVault, revenueSink, the fee settings, the hook allowlist, pausing and the rescue functions:

<https://github.com/Hoodlock/hoodlock-lp-locker-audit/blob/5af08550d606d6f7a458abbef7c09915df27def/src/HoodLockLPLocker.sol#L316-L319>

```
function adminCollect(uint256[] calldata ids) external nonReentrant onlyAdmin {
    require(ids.length ≤ MAX_BATCH, "batch too large");
    _collectEach(ids, true);
}
```

Running adminCollect on a schedule needs a bot, so the full admin key has to live on a server. Two smaller points add to this:

- returnStrayPosition sends a stray position NFT to any to the admin picks, since the contract cannot know who sent it.
- A transferAdmin nomination never expires, so a forgotten pendingAdmin can still take the role later.

IMPACT

If the bot key leaks, the attacker gets every admin power: redirect protocol revenue through feeVault and revenueSink, pause new locks, and send stray tokens and stray position NFTs to any address. Locked positions are not affected.

RECOMMENDATION

Use a multisig as admin, and add a separate keeper role that can only call adminCollect. Cancel unused nominations by calling transferAdmin with address(0).

L-02

LOW

● FIXED

`_tryTransfer` can revert instead of returning `false`

`_payCut` relies on `_tryTransfer` returning `false` on a failed transfer, so it can park the cut in `pendingCut`. But `_tryTransfer` decodes the return data with `abi.decode`, which reverts when a token returns something other than empty data or a valid `bool` (for example the value 2, or fewer than 32 bytes):

[https://github.com/Hoodlock/hoodlock-lp-locker-](https://github.com/Hoodlock/hoodlock-lp-locker-audit/blob/5af08550d606d6f7a458abbef7c09915df27def/src/HoodLockLPLocker.sol#L598-L601)

[audit/blob/5af08550d606d6f7a458abbef7c09915df27def/src/HoodLockLPLocker.sol#L598-L601](https://github.com/Hoodlock/hoodlock-lp-locker-audit/blob/5af08550d606d6f7a458abbef7c09915df27def/src/HoodLockLPLocker.sol#L598-L601)

```
function _tryTransfer(address token, address to, uint256 amount) private returns (bool) {
    (bool ok, bytes memory data) = token.call(abi.encodeWithSelector(0xa9059cbb, to, amount));
    return ok && (data.length == 0 || abi.decode(data, (bool))); // @audit reverts on malformed
    data
}
```

IMPACT

When such a token refuses the cut, the whole collect reverts instead of parking it, and the owner cannot collect the lock's fees. Withdraw is unaffected, and only tokens with non-standard return data are affected.

RECOMMENDATION

Treat malformed return data as a failed transfer:

[https://github.com/Hoodlock/hoodlock-lp-locker-](https://github.com/Hoodlock/hoodlock-lp-locker-audit/blob/5af08550d606d6f7a458abbef7c09915df27def/src/HoodLockLPLocker.sol#L600)

[audit/blob/5af08550d606d6f7a458abbef7c09915df27def/src/HoodLockLPLocker.sol#L600](https://github.com/Hoodlock/hoodlock-lp-locker-audit/blob/5af08550d606d6f7a458abbef7c09915df27def/src/HoodLockLPLocker.sol#L600)

```
- return ok && (data.length == 0 || abi.decode(data, (bool)));
+ return ok && (data.length == 0 || (data.length >= 32 && abi.decode(data, (uint256)) == 1));
```

L-03

LOW

● FIXED

Owed ETH is stuck if the recipient cannot receive ETH

When a native ETH push fails, `_send` credits `owedEth[to]`. That credit can only be paid to the same address: `claimOwedEth` pays `msg.sender` and `pushOwedEth` pays `who`. There is no way to choose another destination:

[https://github.com/Hoodlock/hoodlock-lp-locker-](https://github.com/Hoodlock/hoodlock-lp-locker-audit/blob/5af08550d606d6f7a458abbef7c09915df27def/src/HoodLockLPLocker.sol#L435-L450)

[audit/blob/5af08550d606d6f7a458abbef7c09915df27def/src/HoodLockLPLocker.sol#L435-L450](https://github.com/Hoodlock/hoodlock-lp-locker-audit/blob/5af08550d606d6f7a458abbef7c09915df27def/src/HoodLockLPLocker.sol#L435-L450)

```
function claimOwedEth() external {
    pushOwedEth(msg.sender);
}
...
function pushOwedEth(address who) public nonReentrant {
    uint256 amount = owedEth[who];
    require(amount > 0, "nothing owed");
    owedEth[who] = 0;
    (bool ok, ) = who.call{value: amount}(""); // @audit always sent to `who`
    require(ok, "eth transfer failed");
    emit EthClaimed(who, amount);
}
```

If the recipient is a contract with no payable `receive` or `fallback`, both functions revert. This is easy to hit, since `transferLockOwnership` sets `feeRecipient` to the new owner. A lock moved to a contract owner that does not accept ETH (for example a vesting or escrow contract) gets its native ETH fees credited there on every `collect` or `adminCollect`.

IMPACT

ETH fees credited to such a recipient are locked forever, since the contract has no way to rescue ETH. This goes against the `pushOwedEth` comment that a recipient is "not left with a credit it can never take".

RECOMMENDATION

Add a function that lets the credited address send its credit to another address. The existing functions stay unchanged:

[https://github.com/Hoodlock/hoodlock-lp-locker-](https://github.com/Hoodlock/hoodlock-lp-locker-audit/blob/5af08550d606d6f7a458abbef7c09915df27def/src/HoodLockLPLocker.sol#L435-L437)

[audit/blob/5af08550d606d6f7a458abbef7c09915df27def/src/HoodLockLPLocker.sol#L435-L437](https://github.com/Hoodlock/hoodlock-lp-locker-audit/blob/5af08550d606d6f7a458abbef7c09915df27def/src/HoodLockLPLocker.sol#L435-L437)

```
function claimOwedEth() external {
    pushOwedEth(msg.sender);
}
+
+ function claimOwedEthTo(address to) external nonReentrant {
+     require(to != address(0), "zero to");
+     uint256 amount = owedEth[msg.sender];
+     require(amount > 0, "nothing owed");
+     owedEth[msg.sender] = 0;
+     (bool ok, ) = to.call{value: amount}("");
+     require(ok, "eth transfer failed");
+     emit EthClaimed(msg.sender, amount);
+ }
```

● INFORMATIONAL SEVERITY · 1 FINDING

I-01

INFORMATIONAL

● FIXED

isUnlocked returns true for non-existent locks

`isUnlocked` compares `block.timestamp` against the stored `unlockTime` without checking that the lock exists:

[https://github.com/Hoodlock/hoodlock-lp-locker-](https://github.com/Hoodlock/hoodlock-lp-locker-audit/blob/5af08550d606d6f7a458abbef7c09915df27def/src/HoodLockLPLocker.sol#L551-L553)

[audit/blob/5af08550d606d6f7a458abbef7c09915df27def/src/HoodLockLPLocker.sol#L551-L553](https://github.com/Hoodlock/hoodlock-lp-locker-audit/blob/5af08550d606d6f7a458abbef7c09915df27def/src/HoodLockLPLocker.sol#L551-L553)

```
function isUnlocked(uint256 id) external view returns (bool) {
    return block.timestamp ≥ _locks[id].unlockTime; // @audit unlockTime is 0 for an unknown
    id
}
```

For an id that was never created (for example `0` or any id at or above `nextLockId`), `unlockTime` is `0`, so the function returns `true`.

IMPACT

Frontends, indexers or integrations that use `isUnlocked` see a lock that does not exist as "unlocked" instead of getting an error. No funds are at risk.

RECOMMENDATION

Revert for unknown ids, the same way `_collect` does. A real lock always has a non-zero `owner` :

[https://github.com/Hoodlock/hoodlock-lp-locker-](https://github.com/Hoodlock/hoodlock-lp-locker-audit/blob/5af08550d606d6f7a458abbef7c09915df27def/src/HoodLockLPLocker.sol#L551-L553)

[audit/blob/5af08550d606d6f7a458abbef7c09915df27def/src/HoodLockLPLocker.sol#L551-L553](https://github.com/Hoodlock/hoodlock-lp-locker-audit/blob/5af08550d606d6f7a458abbef7c09915df27def/src/HoodLockLPLocker.sol#L551-L553)

```
function isUnlocked(uint256 id) external view returns (bool) {
+   require(_locks[id].owner ≠ address(0), "no lock");
    return block.timestamp ≥ _locks[id].unlockTime;
}
```



AUDIT COMPLETE

Stay secure.

Thank you for choosing SBSecurity.

WEBSITE

sbsecurity.net

TELEGRAM

[@sbsecurity_audits](https://t.me/sbsecurity_audits)

REPORT

HoodLock · October 2026